

# SYNTH-JEPA: JOINT EMBEDDING PREDICTION FOR RENDERER-FREE SYNTHESIZER PARAMETER SEARCH

Ben Hayes<sup>1</sup>, Haokun Tian<sup>1,2</sup>, Stefan Lattner<sup>1</sup>

<sup>1</sup>Sony Computer Science Laboratories, Paris, France

<sup>2</sup>Queen Mary University of London, UK

## ABSTRACT

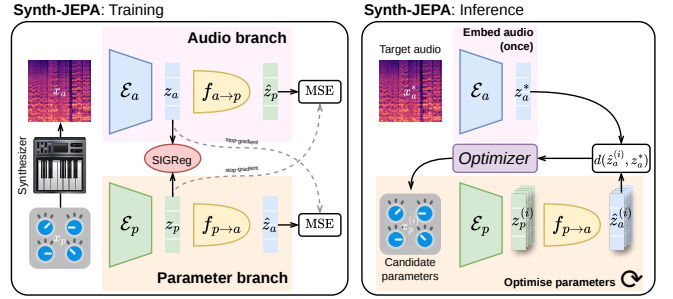
Sound matching can be formulated as optimizing synthesizer parameters against an audio-domain objective. However, objectives derived from generic audio representations are often difficult to optimize, while direct search requires rendering every candidate. We introduce *Synth-JEPA*, which learns mutually predictive audio and parameter representations from paired synthesizer data. At inference, candidate parameters are scored directly in this learned space, yielding a renderer-free objective whose audio geometry is shaped by parameter correspondences rather than generic audio similarity. We evaluate Synth-JEPA on Surge XT using held-out synthesizer sounds and out-of-domain NSynth and FSD50K targets, against inverse models, direct search, and learned proxy objectives. Synth-JEPA outperforms all baselines in-domain and remains competitive out-of-domain. Its matching quality continues to improve with additional test-time search, allowing compute to be traded for match quality. In pairwise listening tests, listeners preferred Synth-JEPA in 85% of trials overall. Together, these results show that an audio representation with a parameter-induced geometry allows synthesizer sound matching to be approached as an effective renderer-free search problem.

**Index Terms**— audio synthesis, sound matching, synthesizer inversion, representation learning, inference-time optimization

## 1. INTRODUCTION

Modern audio synthesizers are intricate systems, combining many methods for sound production and manipulation with large numbers of interacting controls. Selecting parameters to approximate a target sound is therefore challenging. Further, many synthesizers can produce similar or equivalent signals from different parameter configurations, meaning the corresponding inverse mapping is generally ill-posed [1].

Existing approaches broadly either optimize synthesizer parameters separately for each target [2–4], or learn an amortized mapping from audio to parameters using supervised estimation [5], conditional generative models [1, 6, 7], differentiable synthesis [8, 9], or reward-based training [10]. Search



**Fig. 1: Synth-JEPA training and inference.**  $\mathcal{E}_a$  and  $\mathcal{E}_p$  are audio and parameter encoders, while  $f_{a \rightarrow p}$  and  $f_{p \rightarrow a}$  are cross-domain predictors. At inference, target audio is encoded once and candidate parameters are optimized directly against the learned objective, without rendering candidate audio during search.

methods evaluate candidate parameters according to an audio-domain objective, but typically require rendering each candidate with the synthesizer. Learned proxies can reduce this cost by predicting audio representations from parameters [11, 12], but search quality then depends both on the accuracy of the proxy and on the choice of representation. In particular, general audio representations are not learned from the relationship between synthesizer parameters and the sounds they produce.

We propose *Synth-JEPA*, a joint audio–parameter predictive model used as an objective for parameter search, which learns mutually predictive representations of paired synthesizer parameters and rendered audio. This makes the learned audio representation depend directly on parameter–audio correspondences. At inference, the target audio is encoded once and candidate parameters are scored using the learned parameter encoder and cross-domain predictor. Search can therefore be conducted directly in the audio embedding space, without any calls to the synthesizer.

Most closely related to our representation-learning setup, Braun and Finkelstein adapt the SLAP EMA-teacher objective [13] to learn joint DX7 audio–parameter embeddings for preset retrieval [14]. Here, we instead use a joint audio–

parameter representation as the objective for iterative parameter search, and compare the SLAP-style EMA objective with SIGReg [15] within the same Synth-JEPA framework.

Audio examples are available online.<sup>1</sup>

## 2. SYNTH-JEPA

Let  $\mathcal{P}$  denote the synthesizer parameter space, which contains both continuous and discrete controls, and let  $\mathcal{S} \subset \mathbb{R}^n$  denote the space of audio signals. We write the synthesizer as  $S : \mathcal{P} \rightarrow \mathcal{S}$ . Our training data consists of pairs  $(x, y)$ , where  $x \in \mathcal{P}$  is sampled from the training parameter distribution and  $y = S(x)$  is the corresponding rendered signal. We sample parameters from a uniform prior and, following prior work [1], map continuous parameters to  $[-1, 1]$  and adopt a one-hot encoding for discrete parameters. The sampled parameter distribution induces a generally non-uniform distribution over  $\mathcal{S}$ , which may therefore affect the geometry learned for search. Future work will explore audio-aware sampling strategies.

### 2.1. Cross-domain predictive learning

Audio and parameters are mapped to separate representations,

$$z_a = \mathcal{E}_a(y), \quad z_p = \mathcal{E}_p(x), \quad (1)$$

with audio and parameter encoders  $\mathcal{E}_a$  and  $\mathcal{E}_p$ , respectively. These are then mapped to the opposite modality using cross-domain predictors,

$$\hat{z}_p = f_{a \rightarrow p}(z_a), \quad \hat{z}_a = f_{p \rightarrow a}(z_p). \quad (2)$$

We train these mappings with the predictive objective

$$\mathcal{L}_{\text{pred}} = \text{MSE}(\hat{z}_p, \text{sg}(z_p)) + \text{MSE}(\hat{z}_a, \text{sg}(z_a)), \quad (3)$$

where  $\text{sg}$  is the stop-gradient operator. This setup is illustrated in Fig. 1.

To prevent collapse we use SIGReg, the regularization strategy proposed in LeJEPA [15], to regularize the encoders' output distributions towards an isotropic Gaussian. We apply SIGReg to each encoder branch independently, following the setup in LeVLJEPA [16]. To explore the sensitivity of the resulting embedding to the choice of anti-collapse mechanism, we also train an *EMA teacher* variant using the joint-embedding objective proposed in SLAP [13] which is used recently for FM parameter-audio joint representation learning [14].

### 2.2. Renderer-free parameter search

Given target audio  $y^*$ , we compute  $z_a^* = \mathcal{E}_a(y^*)$  once. Candidate parameters are then scored by predicting their representation in the audio space,

$$D_{\text{JEPA}}(y^*, x) = \text{MSE}(z_a^*, f_{p \rightarrow a}(\mathcal{E}_p(x))), \quad (4)$$

<sup>1</sup><https://benhayes.net/synth-jepea>

which, importantly, doesn't require a synthesizer call. We thus estimate

$$\hat{x} = \arg \min_{x \in \mathcal{P}} D_{\text{JEPA}}(y^*, x). \quad (5)$$

Since  $D_{\text{JEPA}}$  is differentiable, Eq. 5 can be solved with gradient-free or gradient-based optimizers. We adopt a hybrid two-stage approach. First, we spend half the evaluation budget evolving a population of 32 candidates with JADE [17]. Mutation and crossover operators are applied to continuous parameters, while categorical parameters are either copied from another candidate or resampled. Second, the eight best candidates as scored by  $D_{\text{JEPA}}$  are further refined by gradient descent on their continuous parameters using Adam with an initial learning rate of 0.1 and cosine decay to zero. During this refinement stage, we prune the candidate pool three times at evenly spaced intervals, each time discarding the worse-performing half according to  $D_{\text{JEPA}}$ . After the final pruning step, the sole remaining candidate is returned.

## 3. EXPERIMENTS

### 3.1. Data and tasks

Following prior work [1], we generate training data using the Surge XT software synthesizer. We adopt a set of 139 active parameters, similar to those selected in [11], but disable audio effects. We disable any modulator settings known to introduce non-determinism (e.g. Sample & Hold). Rather than fixing a dataset size, we render in-domain audio online during training. Audio is rendered at 44.1 kHz in stereo with a duration of 3.0 seconds. Audio encoder inputs are log-Mel spectrograms with a 25 ms window and 10 ms hop. To avoid hand-tuned input normalization, we estimate channel-wise mean and variance over the first 8k training spectrograms using Welford's online algorithm [18] and freeze the resulting statistics for the remainder of training.

We first evaluate on held-out *in-domain* targets generated by the same synthesizer and rendering protocol. We then evaluate general sound matching on the test splits of NSynth [19] and FSD50K [20], trimmed to the first 3s of each clip, resampled to 44.1 kHz, and upmixed to stereo.

### 3.2. Models and baselines

Synth-JEPA's audio encoder  $\mathcal{E}_a$  is a transformer operating on stereo log-Mel spectrograms (128 bands, 25 ms window, 10 ms hop), projected via 1D convolution to 150 tokens. A learned summary token is concatenated to the sequence and its output is taken as  $z_a \in \mathbb{R}^{512}$ . The parameter encoder  $\mathcal{E}_p$  represents each parameter with a linear projection plus a parameter-specific bias. A Perceiver bottleneck [21] with 32 learned latents cross-attends to these tokens, followed by eight self-attention blocks. Averaging the final outputs gives  $z_p \in \mathbb{R}^{512}$ . Both encoders use an internal dimension of 512,

**Table 1:** Sound matching results for in-domain Surge XT presets drawn from the parameter prior, and for out-of-domain NSynth and FSD50K targets, 1 024 targets per set. Bold and underlining mark the best and second-best result in each column.

| Family          | Method                           | In-domain presets |              |              | NSynth       |              |              | Freesound (FSD50K) |              |              | Renders / target | Parameters |
|-----------------|----------------------------------|-------------------|--------------|--------------|--------------|--------------|--------------|--------------------|--------------|--------------|------------------|------------|
|                 |                                  | MSS ↓             | wMFCC ↓      | CLAP ↑       | MSS ↓        | wMFCC ↓      | CLAP ↑       | MSS ↓              | wMFCC ↓      | CLAP ↑       |                  |            |
| Synth-JEPA      | SIGReg                           | <b>8.44</b>       | <b>11.07</b> | <b>0.793</b> | <b>10.25</b> | <u>15.54</u> | <b>0.465</b> | <u>13.59</u>       | 14.81        | <u>0.250</u> | 0                | 53.3M      |
|                 | EMA Teacher                      | 11.52             | 15.29        | 0.625        | 14.02        | 19.33        | 0.323        | 15.39              | 17.33        | 0.179        | 0                | 53.3M      |
| Amortized       | AST Regression                   | 17.94             | 23.16        | 0.499        | 28.06        | 30.64        | 0.104        | 21.48              | 26.56        | 0.086        | 0                | 25.0M      |
|                 | Flow Matching                    | 12.75             | 14.78        | 0.707        | 16.67        | 21.24        | 0.356        | 17.94              | 18.84        | 0.204        | 0                | 44.0M      |
| Renderer search | Log-Mel distance                 | 15.86             | <u>12.17</u> | 0.553        | 17.81        | <b>14.10</b> | 0.199        | 14.55              | <b>11.61</b> | 0.178        | 2048             | —          |
|                 | Embedding distance (mn20)        | 12.90             | 16.43        | 0.730        | 14.61        | 25.05        | 0.387        | 16.12              | 21.41        | 0.249        | 2048             | 17.9M      |
|                 | Learned $\mathcal{E}_a$ distance | <u>9.48</u>       | 13.15        | 0.729        | 12.31        | 17.30        | 0.360        | 14.49              | 16.88        | 0.208        | 2048             | 17.9M      |
| Proxy search    | Embedding (mn20) proxy           | 12.56             | 15.77        | <u>0.768</u> | 13.65        | 23.98        | <u>0.436</u> | 15.44              | 19.79        | <b>0.261</b> | 0                | 31.8M      |
|                 | Audio (log-Mel) proxy            | 10.75             | 12.74        | 0.699        | <u>11.03</u> | <b>14.10</b> | 0.350        | <b>12.18</b>       | <u>13.87</u> | 0.216        | 0                | 31.8M      |

with 8 attention heads. The predictors  $f_{p \rightarrow a}$  and  $f_{a \rightarrow p}$  are three-block residual MLPs with hidden width 1024 and linear output heads. The full model has 53M parameters. To understand the effect of the SIGReg anti-collapse term, we also train a version of Synth-JEPA which instead uses an EMA teacher model [13, 14]. We train for 1M steps with batch size 64, using AdamW with a learning rate of  $3 \times 10^{-4}$  with a warmup-stable-decay scheduler [22], and a weight decay of 0.05.

We compare against two amortized predictive baselines. The first performs regression with an audio spectrogram transformer [5], while the second uses a symmetry-aware parameter domain flow matching model [1]. We adopt the same training recipe for both.

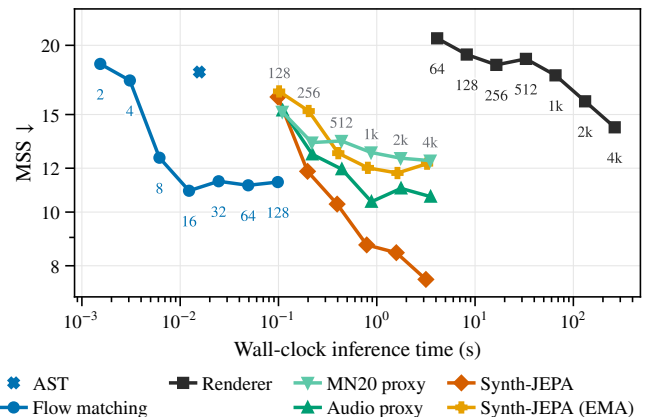
We also compare to direct, or “renderer-in-the-loop” (RITL) search, in which each candidate is rendered by the synthesizer. For this setup, we score candidates using (i) log-Mel  $L_1$  distance; (ii) a pretrained audio embedding (we adopt the mn20 variant of EfficientAT [23], following [12]); or (iii) our trained Synth-JEPA audio encoder  $\mathcal{E}_a$ .

To disentangle the effect of amortization from that of parameter-induced embedding geometry, we also train two neural proxies which predict audio representations from parameter input. Like the RITL baselines, the first directly predicts a log-Mel spectrogram of the synthesizer’s audio. The second follows the work of Combes et al. [12] and predicts a pretrained audio embedding (EfficientAT mn20 [23]).

As the synthesizer is not differentiable, all RITL variants are restricted to using only JADE [17] for their full search budget, while Synth-JEPA and the proxy objectives additionally permit gradient-based refinement (Sec. 2.2). The  $\mathcal{E}_a$  RITL variant thus helps isolate the effect of embedding geometry from any benefit from gradient descent.

### 3.3. Evaluation

All metrics are computed after rendering the returned parameters. We report multi-scale spectral distance (MSS), warped MFCC distance (wMFCC), following [1], and cosine similar-



**Fig. 2:** Sound matching performance when scaling the number of model evaluations (search budget / ODE solver steps). Wall-clock time reported per-target on one RTX 3090. Labels indicate number of model/objective evaluations per target.

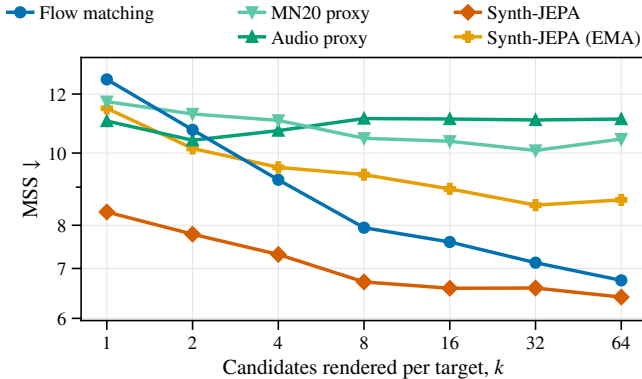
ity between fixed CLAP [24] embeddings of the target and reconstruction. Wall-clock times are measured per target on an RTX 3090.

## 4. RESULTS

### 4.1. Sound-matching performance

Table 1 shows sound matching results for all methods, with a budget of 2048 objective evaluations for search-based methods, and 20 ODE steps for flow matching. In-domain, Synth-JEPA (SIGReg) obtains the best result across all metrics. Its performance remains strong outside the training distribution, achieving best or second-best performance on all metrics except wMFCC, where log-Mel-based objectives lead.

The learned  $\mathcal{E}_a$  representation gives the strongest renderer-in-the-loop MSS results across all datasets, indicating that the parameter-induced embedding geometry itself benefits search. Synth-JEPA (SIGReg) also broadly outperforms both proxy baselines in-domain and on NSynth, though its



**Fig. 3:** Sound matching performance with inference-time best-of- $k$  selection of rendered audio.

performance is more mixed on Freesound’s wider audio distribution.

SIGReg outperforms the EMA-teacher variant on every dataset and metric. This suggests that the choice of anti-collapse mechanism exerts a meaningful influence on the resulting optimization geometry.

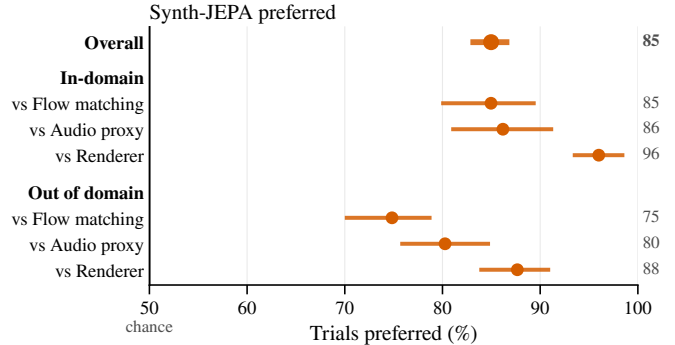
#### 4.2. Inference-time scaling

Fig. 2 highlights the difference between amortized inference and target-specific optimization. Flow matching improves rapidly with additional ODE solver steps but then largely saturates beyond 16 steps. Synth-JEPA, despite requiring more time to achieve flow matching’s performance, continues to improve beyond this as the search budget increases. Other search methods also benefit from greater budgets, but fail to match the performance of Synth-JEPA. This indicates that additional test-time compute is more effectively exploited through target-specific search than through increasingly accurate amortized inference.

We can also scale test-time compute through an optional final best-of- $k$  selection of rendered candidates. Specifically, we render a number of candidates through the synthesizer (meaning this approach is not strictly renderer-free) and select the one that most closely matches the target, according to the  $L_1$  distance between log-Mel spectrograms. The effects of this are illustrated in Fig. 3. Both flow matching and Synth-JEPA improve with best-of- $k$  search, but Synth-JEPA remains the strongest approach at all tested values of  $k$ . Flow matching shows a steeper improvement over its  $k = 1$  baseline, suggesting that its conditional distribution contains high-quality solutions that are not reliably achieved with a single sample.

#### 4.3. Perceptual evaluation

We conducted an online reference-conditioned pairwise listening test comparing Synth-JEPA with flow matching, log-



**Fig. 4:** Pairwise listening tests. Participants compared a reference to two sound matching attempts and selected the closest. Points show proportion of trials in which Synth-JEPA was preferred, with 95% confidence intervals computed by bootstrap resampling of participants.

Mel RITL search, and the log-Mel proxy. Eighteen listeners judged 16 targets (8 in-domain, 4 NSynth, 4 FSD50K) across 3 baselines, giving 48 pairwise trials per participant. Trial order and A/B assignment were randomized. The stimuli in each trial were collectively normalized to a shared loudness of  $-23$  LUFS, thus preserving relative level differences between the reference and reconstructions. Listeners were asked to use headphones or high quality speakers. Four attention-check trials, containing the ground truth as an option, were interleaved with the real trials. All participants passed these checks.

Listeners preferred Synth-JEPA in 85.1% of trials overall (Fig. 4), with preference remaining high both in-domain (89.1%) and out-of-domain (81.0%). Preference was strongest against RITL and weakest against flow matching.

## 5. CONCLUSION

We introduced Synth-JEPA, which learns mutually predictive audio and parameter representations from paired synthesizer data and uses them directly as a renderer-free objective for parameter search. Our results show that it consistently outperforms parameter prediction baselines. Comparisons within the renderer-in-the-loop and differentiable renderer-free settings indicate that Synth-JEPA’s particular embedding space, learned directly from audio–parameter correspondences, is a more effective search objective than generic audio representations. Accordingly, listeners preferred sound matches produced by Synth-JEPA in 85% of trials in a pairwise listening test.

While the model we present here was not trained for non-synthesizer audio, the generality of the JEPA framework permits the inclusion of various self-supervised learning objectives, thus allowing further methods for shaping the embedding geometry. We plan to explore this in future work, as well as the extension of the framework to further synthesizers.

## 6. ACKNOWLEDGEMENTS

Haokun Tian is supported by the EPSRC UKRI Centre for Doctoral Training in Artificial Intelligence and Music (grant number EP/S022694/1).

## 7. COMPLIANCE WITH ETHICAL STANDARDS

All participants provided informed consent for their participation. Data was collected in line with the principles of the Declaration of Helsinki.

## 8. REFERENCES

- [1] Ben Hayes, Charalampos Saitis, and György Fazekas, “Audio synthesizer inversion in symmetric parameter spaces with approximately equivariant flow matching,” in *Proceedings of the 26th ISMIR Conference*, Daejeon, Korea, 2025.
- [2] Andrew Horner, James Beauchamp, and Lippold Haken, “Machine Tongues XVI: Genetic Algorithms and Their Application to FM Matching Synthesis,” *Computer Music Journal*, vol. 17, no. 4, pp. 17–29, 1993.
- [3] Matthew John Yee-King, Leon Fedden, and Mark d’Inverno, “Automatic Programming of VST Sound Synthesizers Using Deep Networks and Other Techniques,” *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 2, no. 2, pp. 150–159, 2018.
- [4] Jordie Shier, George Tzanetakis, and Kirk McNally, “SpiegelLib: An automatic synthesizer programming library,” *Journal of the Audio Engineering Society*, 2020.
- [5] Fred Bruford, Frederik Blang, and Shahan Nercessian, “Synthesizer Sound Matching Using Audio Spectrogram Transformers,” in *Proceedings of the 27th International Conference on DAFX*, Guildford, Surrey, 2024.
- [6] Philippe Esling, Naotake Masuda, Adrien Bardet, Romeo Despres, and Axel Chemla-Romeu-Santos, “Flow Synthesizer: Universal Audio Synthesizer Control with Normalizing Flows,” *Applied Sciences*, vol. 10, no. 1, pp. 302, 2020.
- [7] Gwendal Le Vaillant, Thierry Dutoit, and Sebastien Dekeyser, “Improving Synthesizer Programming From Variational Autoencoders Latent Space,” in *2021 24th International Conference on DAFX*, Vienna, Austria, 2021, pp. 276–283, IEEE.
- [8] Naotake Masuda and Daisuke Saito, “Synthesizer sound matching with differentiable DSP,” in *Proceedings of the 22nd International Society for Music Information Retrieval Conference*, Online, Nov. 2021, pp. 428–434.
- [9] Yuting Yang, Zeyu Jin, Connelly Barnes, and Adam Finkelstein, “White box search over audio synthesizer parameters,” in *Proceedings of the 24th ISMIR Conference*, Milan, Italy, 2023.
- [10] Wonchul Shin and Kyogu Lee, “SynthRL: Cross-domain synthesizer sound matching via reinforcement learning,” in *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI-25)*, Montreal, Canada, Aug. 2025, pp. 10162–10170.
- [11] Oren Barkan, Moshe Laufer, Shlomi Shvartzman, Noy Uzzrad, Almog Elharar, and Noam Koenigstein, “InverSynth II: Sound matching via self-supervised synthesizer-proxy and inference-time finetuning,” in *Proceedings of the 24th ISMIR Conference*, Milan, Italy, 2023.
- [12] Paolo Combes, Stefan Weinzierl, and Klaus Obermayer, “Neural Proxies for Sound Synthesizers: Learning Perceptually Informed Preset Representations,” *Journal of the Audio Engineering Society*, vol. 73, no. 9, pp. 561–577, 2025.
- [13] Julien Guinot, Alain Riou, Elio Quinton, and George Fazekas, “SLAP: Siamese Language-Audio Pretraining without negative samples for Music Understanding,” in *Proceedings of the 26th ISMIR Conference*, Daejeon, Korea, 2025.
- [14] David Braun and Adam Finkelstein, “FM Synthesizer Audio-Parameter Shared Embeddings,” 2026, arXiv:2608.18226 [cs.SD].
- [15] Randall Balestriero and Yann LeCun, “LeJEPa: Provable and Scalable Self-Supervised Learning Without the Heuristics,” 2025, arXiv:2511.08544 [cs.LG].
- [16] Lukas Kuhn, Giuseppe Serra, Randall Balestriero, and Florian Buettner, “LeVLJEPa: End-to-End Vision-Language Pretraining Without Negatives,” 2026, arXiv:2607.00784 [cs.CV].
- [17] Jingqiao Zhang and Arthur C. Sanderson, “JADE: Adaptive Differential Evolution With Optional External Archive,” *IEEE Transactions on Evolutionary Computation*, vol. 13, no. 5, pp. 945–958, 2009.
- [18] B. P. Welford, “Note on a method for calculating corrected sums of squares and products,” *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.
- [19] Jesse Engel, Cinjon Resnick, Adam Roberts, Sander Dieleman, Mohammad Norouzi, Douglas Eck, and Karen Simonyan, “Neural audio synthesis of musical notes with WaveNet autoencoders,” in *Proceedings of the 34th ICML - Volume 70*, Sydney, Australia, 2017, pp. 1068–1077.
- [20] Eduardo Fonseca, Xavier Favory, Jordi Pons, Frederic Font, and Xavier Serra, “FSD50K: An Open Dataset of Human-Labeled Sound Events,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 30, pp. 829–852, 2022.
- [21] Andrew Jaegle, Felix Gimeno, Andy Brock, Oriol Vinyals, Andrew Zisserman, and Joao Carreira, “Perceiver: General Perception with Iterative Attention,” in *Proceedings of the 38th ICML*, 2021, pp. 4651–4664, PMLR.
- [22] Kaiyue Wen, Zhiyuan Li, Jason Wang, David Hall, Percy Liang, and Tengyu Ma, “Understanding Warmup-Stable-Decay Learning Rates: A River Valley Loss Landscape View,” *ICLR*, vol. 2025, pp. 42840–42885, 2025.
- [23] Florian Schmid, Khaled Koutini, and Gerhard Widmer, “Efficient large-scale audio tagging via transformer-to-CNN knowledge distillation,” in *Proc. ICASSP*, 2023.
- [24] Yusong Wu, Ke Chen, Tianyu Zhang, Yuchen Hui, Taylor Berg-Kirkpatrick, and Shlomo Dubnov, “Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation,” in *Proc. ICASSP*, 2023.